

NP-Vollständigkeit

RECAP: NP-Vollständigkeit

- Um zu zeigen, dass L NP-vollständig ist, muss
 - $L \in \text{NP}$ und
 - L muss NP-hart sein
- Zum Beweis der NP-Härte von L_2 nutzen wir die polynomielle Reduktion eines bereits als NP-hart bekannten Problems L_1 auf das neue Problem L_2 ($L_1 \leq_p L_2$).
- Wichtig: die Reduktion darf nur polynomiellen Aufwand besitzen.

Primzahlproblem

- Primzahlproblem
 - **Eingabe:** Eine natürliche Zahl $k \in \mathbb{N}$.
 - **Ausgabe:** Ist k eine Primzahl?
- Es war lange bekannt, dass dieses Problem in NP liegt (der Beweis ist jedoch nicht offensichtlich). Im Jahr 2002 wurde dann von Agrawal, Kayal, Saxena gezeigt, dass dieses Problem sogar in Polynomzeit lösbar ist.
- Der derzeit beste bekannte Algorithmus hat Laufzeit $O(n^6)$. Daher werden für konkrete Anwendungen (vor allem in der Kryptographie) noch randomisierte Primzahltests verwendet.

Faktorisierung

- Während inzwischen bekannt ist, dass es in Polynomialzeit möglich ist, zu bestimmen, ob eine Zahl eine Primzahl ist, ist es anscheinend schwieriger, die **Faktorisierung** einer Zahl k in Primfaktoren zu bestimmen.
- Die Aufgabe, eine Zahl k in Primfaktoren zu zerlegen, ist eine Funktion. Wir benötigen daher eine Entscheidungsversion des Faktorisierungsproblems.
- Faktorisierung
 - **Eingabe:** Zwei Zahlen $k, r \in \mathbb{N}$.
 - **Ausgabe:** Besitzt k einen Faktor $s \neq 1$, der kleiner als r ist?

Faktorisierung

- Wenn man dieses Problem lösen könnte, dann könnte man durch binäre Suche auch den kleinsten Faktor von k bestimmen und damit k faktorisieren.
- Der Status dieses Problems ist nicht geklärt. Es ist offensichtlich in NP (Faktorisierung von k raten, überprüfen und den kleinsten Faktor mit r vergleichen). Es ist bisher jedoch weder bekannt, ob es in P liegt, noch ob es NP-vollständig ist.

Faktorisierung

- Anwendungen in der Kryptographie:
 - Die Schwierigkeit, das Faktorisierungsproblem zu lösen, ist die Grundlage einiger kryptographischer Verfahren (z.B. RSA).
 - Faktorisierungsproblem dazu besser geeignet als viele als NP-vollständig bekannte Probleme
 - Bestimmte Zahlen (insbesondere Produkte zweier großer Primzahlen) sind *immer* schwer zu faktorisieren.
 - Bei bekannten NP-vollständigen Problemen gibt es dagegen oft viele leicht zu lösende Instanzen.
- Es gibt Algorithmen für Quantencomputer (Algorithmus von Shor), die das Faktorisierungsproblem effizient lösen könnten

Graphisomorphie

- Graphisomorphie
 - **Eingabe:** Zwei ungerichtete Graphen $G_1 = (V_1, E_1)$, $G_2 = (V_2, E_2)$
 - **Ausgabe:** Sind die Graphen G_1 , G_2 isomorph? Das heißt, gibt es eine bijektive Abbildung $f: V_1 \rightarrow V_2$, so dass $\{v_1, v_2\} \in E_1$ genau dann, wenn $\{f(v_1), f(v_2)\} \in E_2$?
- Der **Status dieses Problems** ist ebenfalls nicht geklärt. Es ist offensichtlich in NP (Abbildung f raten). Es ist bisher jedoch weder bekannt, ob es in P liegt, noch ob es NP-vollständig ist.

Subgraphisomorphie

Folgendes Problem ist jedoch NP-vollständig.

- Subgraphisomorphie
- **Eingabe:** Zwei ungerichtete Graphen $G_1 = (V_1, E_1)$, $G_2 = (V_2, E_2)$
- **Ausgabe:** Hat G_1 einen Teilgraphen, der isomorph zu G_2 ist?
- Dieses Problem ist relevant für Mustererkennung und Auffinden von komplexen Strukturen.
- Das Cliques-Problem ist übrigens ein Spezialfall der Subgraphisomorphie.



Komplexitätsklassen

Zeitbeschränkung P, NP

- Bislang haben wir zeitbeschränkte Rechenmodelle (Turingmaschinen) betrachtet.
- P enthält die Probleme, die wir in polynomieller Zeit („effizient“) lösen können.
- NP enthält diejenigen Probleme, die wir in polynomieller Zeit auf nicht-deterministischen Maschinen lösen können.
- Unter $P \neq NP$ können wir nicht alle Probleme aus NP effizient lösen, insbesondere die schwierigsten unter den Problemen aus NP, die NP-vollständigen Probleme, haben keinen Polynomialzeitalgorithmus.

Zeitbeschränkung

- Probleme, welche gegenseitig in Polynomialzeit aufeinander reduzierbar sind, sieht man oft als gleich schwer an.
- Dies partitioniert die Menge aller Probleme.
- Die Menge der NP-vollständigen Probleme bildet eine solche Äquivalenzklasse.
- Die Menge P der in Polynomialzeit lösbaren Probleme bildet eine weitere Äquivalenzklasse.
- Die Menge NP der nichtdeterministisch in Polynomialzeit lösbaren Probleme bildet (vermutlich) *keine* Äquivalenzklasse, da es sowohl P als auch die NP-vollständigen Probleme enthält.

Platzbeschränkung: SPACE

- Neben zeitbeschränkten Turingmaschinen betrachtet man auch platzbeschränkte Turingmaschinen.

- Definition: Platzbeschränkte DTM und akzeptierte Sprachen
 - Sei $f : \mathbb{N} \rightarrow \mathbb{N}$ eine (totale) Funktion. Die Klasse $\text{SPACE}(f(n))$ besteht aus allen Sprachen L , für die es eine deterministische Mehrband-Turingmaschine M gibt mit

$$L = L(M) \text{ und } \text{space}_M(x) \leq f(|x|) \text{ für alle Wörter } x.$$
 - Dabei gibt $\text{space}_M(x)$ die maximale Länge einer Konfiguration (ohne die äußeren Leerzeichen) an, die bei der Rechnung von M auf x vorkommt (= Maximalanzahl der benötigten Felder auf den Bändern).

Platzbeschränkung

- Analog definiert man für nichtdeterministische Turingmaschinen $\text{NSPACE}(f(n))$.
- Typ-1-Sprachen
 - Die Typ-1-Sprachen (erzeugt durch monotone Grammatiken) sind genau die Sprachen, die durch eine nichtdeterministische Turingmaschine mit linearer Platzbeschränkung akzeptiert werden können.
 - Das heißt, $\text{NSPACE}(n)$ ist genau die Menge der Typ-1-Sprachen.

Platzbeschränkung : PSPACE

- Wir definieren nun die Klasse aller Sprachen, die von einer **deterministischen Turingmaschine mit polynomialer Platzbeschränkung** akzeptiert werden können.
- Definition: Komplexitätsklasse PSPACE
 - $PSPACE = \bigcup_{p \text{ Polynom}} SPACE(p(n))$

Platzbeschränkung : NP vs. PSPACE vs. NPSPACE

- **Satz von Savitch:**
 - $PSPACE = NPSPACE$
- Die Klasse verändert sich nicht, wenn wir zwischen deterministischen und nicht-deterministischen Turingmaschinen wechseln.
- Die Probleme in NP sind mit polynomiell großem Speicherplatz auf einer deterministischen TM lösbar, da sich der Kopf einer Turingmaschine pro Zeitschritt nur eine Position bewegen kann, wenn wir alle Wahlmöglichkeiten des Zertifikates simulieren.
- Also gilt $NP \subseteq PSPACE$.
- Außerdem liegt jede Typ-1-Sprache in PSPACE
 - $Typ\ 1 = NSPACE(n) \subseteq NPSPACE = PSPACE$

PSPACE

- Es gibt wieder eine Klasse von PSPACE-vollständigen Problemen, welche (vermutlich) eine Äquivalenzklasse nach obiger Äquivalenzrelation bilden.
 - Beispiel: Erzeugt eine Grammatik alle Wörter?
 - Beispiel: Quantified Boolean Formula Problem ($\exists x_1 \forall x_2 \dots ((x_1 \vee x_2) \wedge \dots)$)?
(Bei SAT gab es nur $\exists$, keine $\forall$)
 - Beispiel: Wer gewinnt ein Spiel (Brettgröße ist Input) mit polynomiell beschränkter Anzahl Züge?

EXPTIME

- Die Klasse der Probleme mit einer Laufzeitschranke $2^{p(n)}$ für ein Polynom p bezeichnen wir als **EXPTIME**.
- Es gilt: $NP \subseteq EXPTIME$.
- Nach Übung: bei Speicherplatzbeschränkung der Größe $s(n)$ nur $2^{O(s(n))}$ viele Konfigurationen für eine Turingmaschine möglich, also Rechenzeit durch $2^{O(s(n))}$ beschränkt. Die Probleme in PSPACE können deshalb in Zeit $2^{p(n)}$ gelöst werden, so dass tatsächlich gilt $PSPACE \subseteq EXPTIME$.
- Es gibt wieder eine Klasse von EXPTIME-vollständigen Problemen, welche (vermutlich) eine Äquivalenzklasse nach obiger Äquivalenzrelation bilden.
 - Beispiel: Hält eine Turingmaschine (Input) in höchstens k Schritten (Input)?
 - Beispiel: Wer gewinnt ein Spiel (Brettgröße ist Input) mit exponentiell beschränkter Anzahl Züge?

Komplexitätslandschaft

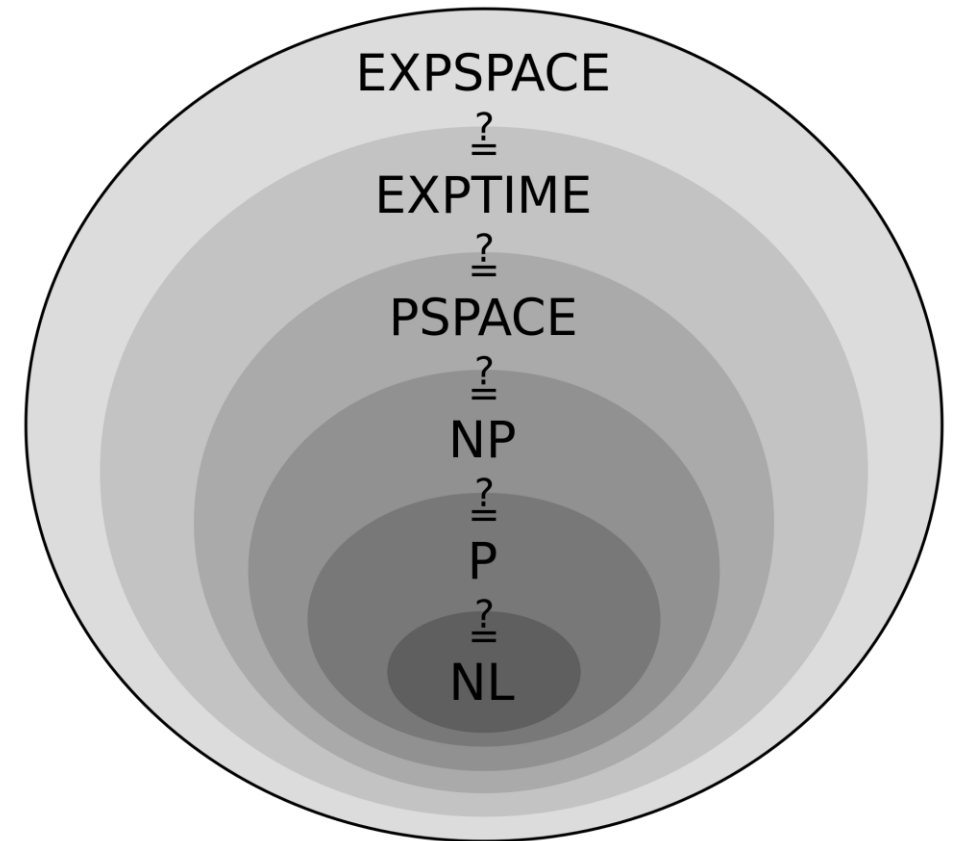
Die folgenden Beziehungen sind bekannt:

$$L \subseteq NL \subseteq P \subseteq NP \subseteq PSPACE \subseteq EXPTIME \subseteq EXSPACE$$

Da P nach dem Zeithierarchiesatz eine echte Teilmenge von $EXPTIME$ ist, muss mindesten eine der Teilmengenbeziehungen $P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$ echt sein. Wahrscheinlich sind alle Inklusionen echt.

L: DTM mit logarithmischer Platzbeschränkung

NL: NTM mit logarithmischer Platzbeschränkung



Komplexitätshierarchie

- Folgende Abbildung beschreibt die Situation unter der verbreiteten Hypothese, dass diese Inklusionen alle echt sind.
- Zwischen P und PSPACE liegt die sogenannte **polynomielle Hierarchie**, deren unterste Stufe durch die Klassen NP und Co-NP gebildet werden und eine (unendliche?) Hierarchie von Klassen bildet. Die Vereinigung dieser Klassen wird oft mit PH bezeichnet.
- Hier enthält **Co-NP** die Komplemente der Sprachen aus NP, also diejenigen Entscheidungsprobleme bei denen die „NEIN-Antworten“ polynomiell verifizierbar sind.
 - Ein Beispiel für ein Co-NP vollständiges Problem ist die Frage, ob eine Boolesche Formel eine Tautologie ist.
- In den unteren Stufen der polynomiellen Hierarchie liegen relevante Probleme

Komplexitätshierarchie

- Es geht weiter mit **NEXPTIME**, **EXPSPACE=NEXPSPACE** (Satz von Savitch), **EXPEXPTIME** (oder auch **2-EXPTIME**) und vielen Zwischenklassen und noch höheren Hierarchien.
- Beispiele für EXPSPACE vollständiges Problem:
 - Umfassen die (schematheoretischen) Nullstellen einer multivarianten polynomiellen Gleichung die Nullstellen eines multivarianten polynomiellen Gleichungssystems
 - Erreichbarkeit in Petrinetzen
- Beispiele für EXPEXPTIME vollständiges Problem:
 - Beispiel: Wer gewinnt ein Spiel (Brettgröße ist Input) mit exponentiell beschränkter Anzahl Züge und nicht vollständig sichtbarem Spielbrett.
- In all diesen Klassen liegen relevante Probleme

Weitere Komplexitätsklassen: Randomisiert

- PP (probabilistic polynomial time)
 - Probabilistisch, Fehler $< \frac{1}{2}$
- BPP (bounded error probabilistic polynomial time)
 - Monte-Carlo-Algorithmen, probabilistisch, Fehler $< \frac{1}{4}$
 - oft als Klasse der effizient lösbaren Probleme angesehen
- RP bzw. co-RP: wie BPP, aber Nein bzw. Ja Antworten sind immer richtig
- ZPP (zero-error probabilistic polynomial time)
 - Randomisierter Algorithmus, aber sicheres Ergebnis
- Ob Inklusionen echt sind ist unbekannt.

